TRABAJO PRÁCTICO

INGENIERÍA DE DATOS I

Integrantes del Grupo Nro:

Uriel Fretes -LU
Lucas Seguin -LU1205392
Ezequiel Leguiza -LU1219991
Santiago Testino Ferrari – LU1214383

Profesor:
Lic. Feichubuinm Alejandro

Cuatrimestre: 2 – Año: 2026

UADE **UNIVERSIDAD ARGENTINA DE LA EMPRESA**
FACULTAD DE INGENIERÍA Y CIENCIAS EXACTAS

ALCANCE:
Check in | Check out *
Reservas *
Lavandería *
Estacionamiento *
Pago y Facturación *
Limpieza *
Empleados *
Proveedores *
Cocina *
Inventario *
Mantenimiento*
Gastos *
Huéspedes *
Habitaciones *
Reclamos de huéspedes *
Gimnasio *
Consumos de huéspedes (minibar, etc)*

Actores:

- **Recepcionista**
- **Gerente del hotel**
- **Personal de limpieza**
- **Personal de cocina**
- **Personal de mantenimiento**
- **Personal de lavandería**
- **Personal del estacionamiento**
- **Personal de administración contable**
- **Jefe de Inventarios**
- **Personal de seguridad**
- **Personal de servicios extra**
- **Proveedores**
- **Huéspedes**

2. Descripción del Sistema

Este sistema de gestión hotelera tiene como objetivo administrar y organizar la información relacionada con las principales actividades de un hotel, mediante una base de datos realizada en SQL Server.
El sistema permite gestionar las reservas de habitaciones, registrar datos de los huéspedes y controlar sus estadía mediante operaciones de check-in y check-out. También, tiene en cuenta la disponibilidad de las habitaciones, sus características y los procesos tanto de limpieza como de mantenimiento.

Además, incluye la administración de consumo realizado por los huéspedes, el control de productos y sus existencias, asignaciones de plazas en el estacionamiento y el registro de pagos y facturas asociados a las reservas.
Por otra parte, también permite almacenar información sobre los empleados, sus horarios, las tareas realizadas y los gastos relacionados con el funcionamiento del establecimiento.
La base de datos tiene en cuenta las restricciones, los procedimientos almacenados y triggers que permite validar las reglas de negocio y automatizar determinadas operaciones.
Asimismo, se contempla el manejo de errores para evitar registros inválidos y operaciones que afecten a la consistencia de los datos.

.

REGLAS DE NEGOCIO

Reservas:

-No se puede cancelar la reserva a menos de 24 horas de la fecha de Check in
-Se debe pagar previo(Mínimo 24 horas) al Check in el 50% por adelantado. El 50% restante al realizar el Check in.
-En el momento del Check out el huésped deberá pagar por todos los servicios utilizados durante la estancia si es que superan el depósito
-No se permiten mascotas
-Los menores de 18 no pueden realizar reservas
-Las reservas se pueden modificar solo a fechas en las que haya disponibilidad
-Si un cliente ingresa mal sus datos o difieren la reserva no se podrá validar
-No se pueden realizar reservas si el hotel está completo
-Los desayunos están incluidos con la reserva
-En caso de que no se use el depósito o sobre plata de este, se devolverá al cliente

CHECK IN / CHECK OUT:

-El huésped deberá realizar el Check Out antes de las 10 hs
-Se cargará una noche más en la cuenta si el huésped no realiza el Check out en la hora acordada
-El Check In deberá realizarse después de las 14hs
-El cliente deberá abonar un depósito de —---- al momento de realizar el Check in
-El huésped no podrá ingresar previo a la hora del Check in a pesar de que esté la habitación liberada
-Si ocurre algo en la habitación reservada y no está disponible durante el Check In, se le informará al huésped y se le dará otra habitación

LAVANDERÍA:

-Se cobrara una comision de —- por el uso servicio
- El servicio tendrá un horario de disponibilidad de 6 am a 7 pm

-Cuando un huésped use la lavandería se le pedirá su ID relacionado al hotel y se le cargará la comisión ahí para ser abonada durante el Check out
-Los pedidos de lavandería van a tener 4 estados: Pendiente — En proceso — Listo — Entregado

LIMPIEZA:
- Se retirarán a la lavandería únicamente las toallas que están ubicadas en el suelo de la habitación
-Durante la limpieza la habitación podrá tener el estado: Limpiándose
- El horario de limpieza va ser de 8:00 am a 16:00 pm
-Si la habitación se encuentra en muy malas condiciones se realizará la limpieza de todas formas pero se impondrá una comisión que se deberá abonar en el Check Out
- Las habitaciones que quedan después del proceso del check-out tiene prioridad de limpieza.
-Si se detectan objetos de valor sobre las camas de la habitación en el momento de la limpieza, el personal no realizará la limpieza de la misma.

ESTACIONAMIENTO:
-El estacionamiento está incluído con cada reserva, pero solo 1 vehículo por reserva.
-Cada plaza de estacionamiento tendrá un único id.
- El estacionamiento estará disponible las 24 horas del día.
-El valet parking está incluído pero será opcional
-El estacionamiento tendrá un límite de —- vehículos simultáneos

PAGO Y FACTURACIÓN
-El cliente deberá especificar su método de pago al momento de realizar la reserva
-Si el pago total es mayor a —-- no se aceptara efectivo como método de pago
- Las facturas deberán incluir el huésped que realizó el pago, el empleado que lo atendió, el método de pago utilizado, y la fecha de la transacción
-Los métodos de pagos que acepta el sistema son: Efectivo, tarjetas nacionales/internacionales y aplicaciones de transferencias bancarias.
-Si previo al Check In se realiza un pago no validado, se cancela la reserva
-Hay un descuento del 5% en las reservas con tarjetas nacionales
EMPLEADOS:
- Cada empleado debe tener un id único.
- Los cambios de puesto se deben registrar en el sistema,
- Los empleados solo pueden contar con un puesto/rol determinado.
- Los horarios y turnos de los empleados deben estar registrados en el sistema
- Si el empleado se da de baja, todo su historial de operaciones quedará conservado en el sistema-
-Los empleados deberán tener una hora de entrada y de salida fijos.
-Los empleados tendrán que registrar sus horarios de entrada y salida
- Los empleados van a tener dos estados: Activos — Desvinculados
-Los empleados pueden tener dos tipos de remuneración: variable (por horas trabajas) y fija (Mensual)
-Cada empleado de limpieza tendrá designado un piso del hotel

PROVEEDORES:

-Debe respetar un horario fijo preestablecido de entrega
-Debe estar registrada el horario en que cada proveedor trae sus productos
- Para registrar un proveedor, se deberá registrar los siguientes datos: razón social, cuit y contactos como teléfono y correo electrónico
-Si un proveedor se retrasa en una entrega de insumos se penalizará con un 5% del valor total del pedido.
-Se deberá registrar qué productos llegaron y su cantidad de cada entrega
-Se deberá registrar que empleado es el que recibe el pedido con su ID de empleado
- No se podrá registrar cantidades superiores a las pedidas.
- Cada orden va tener diferentes estado para verificar que la situación en la que se encuentra: SOLICITADO — EN PROCESO — ENTREGADO
- Las cantidades solicitadas mayor a cero.

COCINA:
-La cocina estará abierta de 6 hs a 23:30hs
-El desayuno estará disponible de 7hs a 10hs
-Los cocineros podrán trabajar como máximo en 2 turnos por día
-Se deberá verificar el estado de los alimentos y notificar al inventario si están en vencidos.
-Los empleados deberán ser notificados de las alergias que pueda poseer un huésped
-Al inicio del día, se deberá notificar si hay algún pedido especial

INVENTARIO:
-Se deberá tener registro de la cantidad restante de cada producto
-Si queda menos del 10% de stock de un producto deberá aparecer una alerta
-Cada producto deberá estar asociado a una categoría y a un ID propio
-Cada categoría deberá tener un ID propio
-Cada mañana se deberá realizar un inventario de los productos restantes

MANTENIMIENTO:
-Cada técnico debe responder a sus solicitudes de mantenimiento dentro de las 24 horas luego de que se crea la solicitud.
-Las áreas de mantenimiento disponibles serán: electricidad, plomería, ascensores, infraestructura, informática.

HUÉSPEDES:
- Cada huésped debe tener un identificador único.
-Para registrar un huésped se deben proporcionar nombre, apellido, dni, fecha de nacimiento y nacionalidad.
- No se puede registrar dos huésped con el mismo dni, número de teléfono o correo electrónico.
- Los huéspedes menores de edad deben estar acompañados por un adulto responsable.
- Los datos personales de los huéspedes sólo podrán ser consultados o modificados por empleados autorizados.
- Una reserva puede estar asociada a varios huéspedes, pero debe identificarse un titular responsable.

GASTOS:

-Se deberá tener registro de todos los gastos del hotel, así como su respectiva fecha y destino/motivo
- Se deberá tener un registro del presupuesto asignado a cada departamento, asimismo con el restante

GIMNASIO:
- El gimnasio está disponible desde 9:00 am hasta las 20:00 pm
- Tanto el ingreso como la salida deberá ser registrada por huésped.
- Los equipos pueden estar “fuera de servicio” y no se podrá utilizar hasta que finalice el mantenimiento correspondiente.
- Cada equipo de gimnasio debe tener un identificador único y un estado operativo.
- El gimnasio contará con una disponibilidad máxima de personas simultáneas que no podrá superarse
-El gimnasio deberá contar con profesores que ayuden a los huéspedes.

CONSUMOS DE HUÉSPEDES:
-Los cobros del minibar de la habitación se agregarán a la cuenta de la habitación.
-Almuerzo, hora del té, y cena se agregan a la cuenta de la habitación.
-La cuenta de la habitación deberá abonarse previo a realizar el check out
-Los precios de los insumos del minibar deben estar detallados en una carta que se ubica en cada habitación.
-Los precios de cada plato en el restaurant deben estar detallados en la carta del mismo.

RECLAMOS DE HUÉSPEDES:
-Cada reclamo tiene la fecha del reclamo registrada, el área, y un huésped asociado
-El reclamo tiene diferentes estados: ABIERTO, CERRADO, CONCLUÍDO.
-Si pasan más de 30 días desde que el reclamo es abierto, se dará por cerrado automáticamente.
-Cuando un reclamo es hecho, se le asignará a un empleado de administración

ENTIDADES

1. **Empleados**: **IdEmpleado,** Nombre, DNI , Correo, Sueldo, horas de trabajo semanales

2. **HorarioEmpleado**: IdEmpleado,HoraEntrada,Horasalida

3. **Habitaciones**: **IdHabitación**, Piso, Estado (ocupada/disponible), IdTipoHabitacion, Limpieza(Sucia-Limpiándose- Limpia).

4. **Reservas**: **IdReserva**, FechaReserva, Estado, Importe, PrecioPorNoche, FechaEntrada, FechaSalida, IdHabitación

5. **Estadía**: IdEstadia, Id Reserva, FechaHoraCheckIn, FechaHoraCheckOut, Estado

6. **Productos**: IdProducto, Nombre, IdCategoria, Stock, Precio

7. **Categorías**: IdCategoria, NombreDeCategoria

8. **Huésped**: **idhuespued**, Nombre, Dni/Pasaporte, Correo, FechaNacimiento, Telefono

9. **ReservasHuespued**: idreserva, idhuespued, esTitular

10. **Vehículos**: id vehiculo, id huésped, Marca, Modelo, Color, Patente, IDplazadeEstacionamiento

11. **Tipos de habitación**: id tipo, nombre tipo, costo, capacidad, cantidadHabitaciones

12. **Limpieza de habitaciones**: idlimpieza, horaini, horafin, fecha, id empleado, idhabitacion

13. **Plaza de estacionamiento**: idPlazadeEstacionamiento, estado

14. Facturas: id factura, id empleado, fecha de emisión, id reserva, tipo de comprobante

15. Pagos: id pago, idreserva, método, importe, Cuotas, descripción, estado

16. Detalle Factura: id Detalle Factura, IdFactura (fk), subtotal, impuesto, estado, total, precio unitario, cantidad

17. Gastos: IdGasto, IdEmpleado, fechaGasto, monto, Estado, CategoriaGasto(Mantenimiento, Insumos, Servicios, Administración)

18. ConsumosdeHuésped: IdHuésped, IdConsumo, fechaConsumo,

19. ProductosporConsumo: idConsumo, IdProducto, cantidad, precioTotal (se puede calcular por fuera)
20. Solicitudes de mantenimiento: idSolicitud, IdEmpleado, fecha, estado, área, descripcion, lugar, Equipo

21. RegistrosDeAsistencia(Empleados): IdEmpleado, horaEntrada, horaSalida, fecha (cambiar hora de entrada y salida que tenga a demas de hora la fecha)

22. ServiciosExtra: idServicio, nombreServicio, precioTotal, Horario (cambiar nombre)

23. ConsumosServiciosExtra: idServicio, idEstadia,

Relaciones

1. Huésped **n** → **m** Reservas. *
2. Empleado 1 → n HorarioEmpleado
3. Empleado 1 → **n** Limpieza de Habitaciones**

4. Empleado 1 → **n** Facturas **
5. Empleado 1 → **n** Gastos**
6. Empleado 1 → **n** Solicitud de mantenimiento***
7. Empleado 1 → **n** Registro de asistencia ***
8. Habitación 1 → n Reservas*
9. Habitación n → 1 TipoHabitación **
10. Reserva 1 → 1 Estadía*
11. ConsumoHuésped n → **m** Productos**
12. Categoría 1 → n producto *
13. Vehículo n → 1 huésped*
14. Vehículo n → 1 plaza de estacionamiento*
15. Limpieza Habitación n → 1 Habitación *
16. Estadia m → **n** ServiciosExtra*
17. factura 1 → n detalle factura*
18. reserva 1 → factura 1*
19. huesped 1 → n consumos huesped*
20. reserva 1 → n pagos

sacamos alergias, tipo remuneracion

NORMALIZACIÓN

1. **Normalizacion de Empleados**
   Empleados (IdEmpleado, Dni, Nombre, Correo, Sueldo, HorasSemanales)
   Clave Primaria: IdEmpleado
   Claves Candidatas: IdEmpleado, Dni y Correo.
   Dependencias funcionales:
   IdEmpleado → DNI, Nombre, Correo, Sueldo, HorasSemanales
   Dni → IdEmpleado
   Correo → IdEmpleado
   Primera Forma Normal (1FN): La relación cumple con 1FN porque todos sus atributos contienen valores atómicos y no presenta grupos repetitivos ni atributos multi valores.
   Segunda Forma Normal (2FN): La relación cumple con 2FN porque cumple con 1FN y no existen dependencias parciales. Sus claves candidatas están formadas por un único atributo.
   Tercera Forma Normal (3FN): La relación cumple porque no se presentan dependencias transitivas que rompan con esta forma normal.

2. **Normalización de HorarioEmpleado**
   HorarioEmpleado(IdHorarioEmpleado, IdEmpleado, HoraEntrada, HoraSalida)
   Clave Primaria: IdHorarioEmpleado
   Clave candidata: IdHorarioEmpleado y IdEmpleado
   Dependencias funcionales:
   IdHorarioEmpleado → IdEmpleado, HoraEntrada,HoraSalida
   IdEmpleado → IdHorarioEmpleado, HoraEntrada, HoraSalida

Primera Forma Normal (1FN): Los atributos son atómicos y no presenta grupos repetitivos.
Segunda Forma Normal (2FN): Hay una sola clave simple primaria. No hay dependencias parciales.
Tercera Forma Normal (3FN): No hay dependencias transitivas que rompan la 3FN.

3. **Normalización de Habitaciones:**
   Habitaciones(IdHabitacion, Estado, Limpieza, Piso, IdTipoHabitacion)
   Clave Primaria: IdHabitacion
   CLAVE CANDIDATAS: sólo IdHabitacion
   Dependencias funcionales:
   IdHabitacion → Estado, Limpieza, Piso, IdTipoHabitacion
   Primera Forma Normal (1FN): Los atributos son atómicos y no hay grupos repetitivos ni con multiples valores
   Segunda Forma Normal (2FN): Como solo hay una clave simple primaria, no existen dependencias parciales
   Tercera Forma Normal (3FN): No se encuentran dependencias transitivas que rompan la 3FN. En Habitaciones no atributos almacenados de TipoHabitacion.

4. **Normalización de Reserva**:
   Reservas(IdReserva, FechaReserva, Estado, Importe,PrecioPorNoche, FechaEntrada, FechaSalida, IdHabitación)
   Clave Primaria: IdReserva
   Clave Candidata: solo IdReserva
   Dependencias funcionales:
   IdReserva → FechaReserva, Estado, FechaEntrada, FechaSalida, Importe, IdHabitacion, PrecioPorNoche.
   Primera Forma Normal (1FN): Todos los atributos contienen valores atómicos y no existen grupos repetitivos.
   Segunda Forma Normal (2FN): La clave Primaria es simple por lo cual no existen dependencias parciales.
   Tercera Forma Normal (3FN):
   Revisar
5. **Normalización de Estadía**
   Estadia(IdEstadia,IdReserva, FechaHoraCheckIn, FechaHoraCheckOut, Estado)
   Clave Primaria: IdEstadia
   Clave Candidata: IdEstadia y IdReserva
   Dependencias funcionales:
   IdEstadia → IdReserva, FechaHoraCheckIn, FechaHoraCheckOut, Estado.
   Pero como la reserva es única para cada estadía.
   IdReserva →IdEstadia, FechaHoraCheckIn, FechaHoraCheckOut, Estado
   Primera Forma Normal (1FN): Los atributos contienen valores atómicos, no hay grupos repetitivos y está identificada de manera única.
   Segunda Forma Normal (2FN): Las dos claves candidatas están formadas por un único atributo.
   Por lo tanto, no existen dependencias funcionales parciales.
   Tercera Forma Normal (3FN): No hay dependencias transitivas que rompan la 3FN

6. **Normalizacion de ReservaHuesped**
   ReservasHuespued(IdReserva, IdHuespued, EsTitular)
   Clave Primaria: IdReserva y IdHuesped
   Clave Candidata: (IdReserva, IdHuesped)
   Dependencia funcional:
   (IdReserva,IdHuesped) → EsTitular
   Primera Forma Normal (1FN): Todos los atributos contienen valores atómicos y no hay atributos multivaluados o grupos repetitivos.
   Segunda Forma Normal (2FN): El atributo EsTitular depende totalmente de la clave compuesta primaria y no únicamente de una.
   Tercera Forma Normal (3FN): No hay dependencias transitivas ya que el único atributo no clave es EsTitular y depende únicamente de la clave primaria compuesta.

7. **Normalizacion de Categorías**
   Categorias(idCategoria, nombredeCategoria)
   Clave primaria: idCategoria
   Primera Forma Normal (1FN): Todos los valores son atomicos(Ninguna categoria posee mas de 1 nombre). Todo se identifica de manera unica con la clave primaria. No hay grupo repetidos, solo hay un valor celda.
   Segunda Forma Normal(2FN): No hay dependencias parcial de la clave primaria debido a que esta no está compuesta, es una clave simple
   Tercera Forma Normal(3FN): No hay dependencias transitivas(En esta caso esta tabla solo posee 2 atributos, una clave principal y otra no, asi que no puede haber transitividad.

8. **Normalizacion de Productos**
   Productos(idProducto, nombreProducto, idCategoria, stock, precio)
   Clave primaria: idProducto
   Clave Candidata: idProducto y Nombre producto(Es un valor atomico para cada producto, no hay 2 productos con el mismo nombre)
   Primera Forma Normal(1FN): Todos los valores son atomicos(Cada producto posee 1 solo nombre, categoria, stock y precio). Con la clave primara (idProducto) podes identificar la totalidad de estos atributos de cada uno. No hay valores redundantes
   Segunda Forma Normal(2FN): No hay dependencias parcial de la clave primaria, debido a que es una clave primaria simple.
   Tercera Forma Normal(3FN): No hay dependencias transitivas, los unicos determinantes son idProducto y nombre, que son primaria y candidata, las demas solamente depende de ellas dos, no son transitivas entre ellas

9. **Normalizacion de Huéspedes:**
   Huespedes(idHuesped,nombreHueped,correoHuesped,fechaNacimiento,numTelefono)
   Clave primaria: idHuesped
   Clave candidata: No hay(correoHuesped y numTelefono podrian ser, pero son valores que admiten ser null)

Primera Forma Normal: Todos los valores son atomicos, debido a que cada huesped únicamente debe proveer un solo telefono y correo(o no proveerlos). Con la clave primaria podes identificar todos los atributos, siendo esta unica para cada individuo.
Segunda Forma Normal(2FN): No hay dependencias parcial de la clave primaria, debido a que es una clave primaria simple.
Tercera Forma Normal(3FN): No hay dependencias transitivas, el unico determinante es idHuesped, no hay transitividad entre ninguna de las demas claves

10. **Normalizacion de Vehiculos:**
    Vehiculos(idVehiculo,idHuesped,
    marcaVehiculo,modeloVehiculo,colorVehiculo,patenteVehiculo)
    Clave primaria: idVehiculo
    Claves candidatas: patenteVehiculo
    Primera Forma Normal(1FN): Todos los valores son atomicos, no hay ningun atributos con mas de un valor
    Segunda Forma Normal(2FN): No hay dependencias parcial de la clave primaria, debido a que es una clave primaria simple.
    Tercera Forma Normal(3FN):No hay dependencias transitivas, las unicas determinantes que hay son de la clave primaria y candidata,

11. **Tipos de habitacion:**
    tiposdeHabitacion(idTipoHabitacion,nombreTipo,costoHabitacion,capacidadHabitacion,cantidadHabitaciones)
    Clave primaria: idTipoHabitacion
    Claves candidatas: idTipoHabitacion
    Primer Forma Normal(1FN): Todos los valores son atomicos, no pueden agarrar mas de un valor, con idTipoHabitacion podes acceder a todos
    Segunda Forma Normal(2FN): No hay dependencias parcial de la clave primaria, debido a que es una clave primaria simple.
    Tercera Forma Normal(3FN): No hay dependencias transitivas, la unica determinante es idTipoHabitacion

12. **LimpiezasdeHabitaciones:**
    LimpiezasdeHabitaciones(idLimpieza,horarioInicioLimpieza,horarioFinLimpieza,fechaLimpieza,idEmpleado, IdHabitacion)
    Clave Primaria: idLimpieza
    Claves candidatas: idLimpieza
    Primer Forma Normal: Todos los valores son atomicos, solo un empleado limpia por habitacion,no hay ningun atributo que pueda tener mas de un valor
    Segunda Forma Normal: No hay dependencias parcial de la clave primaria, debido a que es una clave primaria simple.
    Tercera Forma Normal: No hay dependencias transitivas, la unica que es determinantes es la clave primaria(que asimismo es la única clave candidata)

13. **PlazadeEstacionamiento:**
    PlazasDeEstacionamiento(IdPlazaDeEstacionamiento, Estado)
    Clave primaria: IdPlazaDeEstacionamienot

Claves candidatas: IdPlazaDeEstacionamiento
Primera Forma Normal: Todos los atributos son atomicos, solo puede haber un valor por atributo
Segunda Forma Normal: No hay dependencias parcial de la clave primaria, debido a que es una clave primaria simple.
Tercera Forma Normal: No hay dependencias transitivas(Posee solo 2 atributos y el 2do depende de la clave primaria.

14. **Facturas:**
    Facturas: IdFactura, FechaDeEmision, TipoDeComprobante, IdEmpleado, IdReserva
    Clave primaria: IdFactura
    Claves candidatas: IdFactura
    Primera Forma Normal: Todos los valores son atómicos, cada factura tiene una fecha, un tipo de comprobante, un empleado y una reserva asociada
    Segunda Forma Normal: Cumple. No hay dependencias parciales de la clave primaria
    Tercera Forma Normal: Decidir si TipoDeComprobante almacena sólo un valor o alguna información además (código, descripción, etc). En principio solo almacena un dato.

15. **Pagos:**
    Pagos: IdPago, IdReserva, Estado, Descripcion, Metodo, Importe, Cuotas
    Clave primaria: IdPago
    Claves candidatas: IdPago
    Primera Forma Normal: Cumple, todos los datos almacenan un solo dato.
    Agregar a las reglas del negocio que solo se puede utilizar un método de pago al pagar.
    Segunda Forma Normal: Cumple, la PK es simple y todos los atributos dependen de esta
    Tercera Forma Normal: En principio cumple, pero hay que marcar que cuando el pago se hace con debito o transferencia se haga en **1 sola cuota y cuando se hace con crédito se pueda hacer en más cuotas**

16. **Detalle Factura:**
    DetalleFactura: IdDetalleFactura, IdFactura (fk), subtotal, impuesto, estado, total, precio unitario, cantidad
    Clave Primaria: IdDetallaFactura
    Claves Candidatas:IdDetalleFactura
    Primera Forma Normal: Cumple, los atributos están atomizados y solo almacenan un dato
    Segunda Forma Normal: Cumple, la clave primaria es simple y no hay dependencias parciales
    Tercera Forma Normal: sacar subtotal, porque se puede calcular con sum, revisar total porque se puede calcular sumando el sum que sacamos del subtotal con el impuesto. **Subtotal depende de Precio unitario y cantidad, total depende de subtotal e impuesto**.

17. **Gastos:**
    Gastos: IdGasto, IdEmpleado, FechaGasto, Monto, Estado, CategoriaGasto
    Clave Primaria: IdGasto
    Claves candidatas:
    Primera Forma Normal: Cumple, todos los datos están atomizados
    Segunda Forma Normal: La clave primaria es simple
    Tercera Forma Normal: Si **CategoriaGasto** almacena sólo el tipo de gasto que es y NADA MAS esta bien, si quieren que almacene alguna descripción más hay que hacer otra tabla.

18. **ConsumosDeHuesped:**
    ConsumosDeHuesped: IdConsumo, IdHuesped, FechaConsumo
    Clave Primaria:
    Claves candidatas:
    Primera Forma Normal: cumple, estan todos los datos atomizados
    Segunda Forma Normal: Cumple, la clave primaria es simple
    Tercera Forma Normal: Cumple, no hay información adicional del huésped en la tabla

19. **VehiculosPlazas:**
    VehiculosPlazas(IdVehiculo,IdPlazaDeEstacionamiento,FechaHasta,FechaDesde)
    Clave Primaria(IdVehiculo,IdPLazaDeEstacionamiento)
    Claves candidatas: Las mismas, son una clave compuesta
    Primera Forma Normal: Todos los atributos son atomicos, en todos entra solamente un valor.
    Segunda Forma Normal: Se cumple, debido a que no hay dependencias parciales en la superclave, fechadesde y hasta dependen del vehiculo y del lote al que fue asignado.
    Tercer Forma Normal: No hay dependencias transitivas, el unico determinante es la clave compuesto, debido a que inclusive cada clave parcial de la superclave no sirven como determinantes, solo la superclave.

10.Consultas SQL

1. **Ver Habitaciones Disponibles**
   Objetivo: Listar Habitaciones disponibles para poder asignarlas a nuevos huéspedes
   Esta consulta muestra todas las habitaciones cuyo estado figura “Disponible”. Indica el tipo de habitación, piso y estado de limpieza. Los resultados se ordenan primero por piso y después por número de habitación, de menor a mayor

2. **Ver las reservas entre 2 fechas**
   Objetivo: consultar las reservas que tienen entradas durante un periodo específico.
   Esta consulta obtiene las reservas cuya fecha de entrada se encuentra entre el 1 y el 30 de noviembre de 2026. Muestra los datos principales de cada reserva y las ordena según su fecha de entrada.

3. **Ver productos con poco stock**
   Objetivo: ver qué productos tienen su stock por debajo de 10 unidades.
   La consulta muestra el nombre, precio y stock de todos los productos que tienen menos de 10 unidades en el almacen

4. **Mostrar Habitaciones y tipo de habitacion**
   Objetivo: mostrar cada habitación con su tipo, capacidad, y si hay una reserva activa o no en la habitación.
   La consulta utiliza INNER JOIN para relacionar tres tablas: Reservas, Habitaciones y TiposdeHabitacion. La consulta muestra todas las habitaciones junto a su tipo de habitación, capacidad y cuando ocurrió la última reserva en esta, incluyendo las que fueron canceladas o las que están vigentes

5. **Ver huespedes asociados a cada reserva**
   Objetivo: ver los datos de los huéspedes que están vinculados a cada reserva
   Utiliza tres tablas relacionadas mediante INNER JOIN: Reservas, ReservasHuespedes y Huespedes. La tabla intermedia permite relacionar las reservas con los huéspedes correspondientes. Usa un ORDER BY para organizar los resultados por número de reserva y nombre del huésped

6. **Consultar estadias y reserva**
   Objetivo: mostrar las estadías reigstrados junto con los datos de las reservas a las que pertenecen
   Utiliza INNER JOIN para relacionar las tablas Estadias y Reservas, utilizando IdReserva como campo de unión. Al final usa un ORDER BY DESC que ordena las estadías desde la fecha de Check-In más reciente hasta la más antigua.

7. **Consultar los conusmos que realizaron los huespedes**
   Objetivo: ver qué consumos realizó cada huésped durante su estadía
   Utiliza varios INNER JOIN para relacionar las tablas ConsumosDeHuesped, Huespedes, ProductosPorConsumo y Productos. Estas relaciones permiten conectar cada consumo con el huésped que lo realizó y los productos incluidos. Ordena los consumos según su fecha de manera descendente.

8. **Cuantas reservas hay por tipo de habitacion**
   Objetivo: contar cuántas reservas corresponden a cada tipo de habitación, permitiendo ver cuales tienen mayor demanda
   Utiliza un INNER JOIN para relacionar los tipos de habitación con las habitaciones y sus reservas. Después usa COUNT() para contar la cantidad de reservas de cada categoría.

9. **Cuanto gasto un huespéd en productos**
   Objetivo: calcular cuánto gastó cada huésped en los productos del hotel que consumió
   Utiliza un INNER JOIN para relacionar huéspedes, consumos y productos. Usa la función SUM() para sumar el resultado de multiplicar la cantidad consumida (Cantidad) por el precio de cada producto e identifica el resultado como TotalGastado

10. **Cuantos ingresos hubo por reservas**
    Objetivo: calcular los ingresos generados solo por las reservas por cada mes
    Utiliza las funciones YEAR() y MONTH() para extraer el año y el mes de la fecha de reserva. Luego, DATEDIFF() calcula la cantidad de noches entre la fecha de entrada y salida. Ese resultado se multiplica por PrecioPorNoche y SUM() suma los importes. GROUP BY agrupa los ingresos por año y mes.

11. **Calcular precio promedio por categoria**
    Objetivo: muestra la cantidad de productos por categoría, su precio promedio, mínimo y máximo
    Utiliza un INNER JOIN para relacionar las tablas Categorias y Productos. Luego aplica diferentes funciones de cálculo: COUNT() cuenta los productos, AVG() calcula el precio promedio, MIN() obtiene el precio más bajo y MAX() el más alto.

12. **Encontrar huespedes que realizaron mas de una reserva**
    Objetivo: ver qué huéspedes realizaron más de una reserva
    Utiliza un INNER JOIN para relacionar los huéspedes con la tabla intermedia ReservasHuespedes. La función COUNT(DISTINCT) cuenta las reservas sin repetir identificadores.

13. **Identificar que productos tienen mayor demanda**
    Objetivo: ver que productos fueron consumidos más de 5 veces
    Utiliza un INNER JOIN para relacionar los productos con sus registros de consumo. La función SUM() suma las cantidades consumidas de cada producto

14. **Clasificar reservas según su estado**
    Objetivo: clasificar las reservas según estan finalizadas, en curso o pendientes, dependiendo de la fechas de entrada y salida
    Utiliza CASE para establecer diferentes condiciones mediante WHEN. Compara las fechas de las reservas con GETDATE(), que obtiene la fecha y hora actual del sistema. CAST() convierte ese valor en formato fecha, sin considerar la hora. Dependiendo de qué condición se cumpla, THEN devuelve una clasificación diferente. ELSE establece la categoría pendiente cuando no se cumple ninguna de las condiciones anteriores.

15. **Encontrar habitaciones que nunca tuvieron reservas**
    Objetivo: ver que habitaciones nunca tuvieron una reserva asignada
    Utiliza NOT EXISTS para comprobar que no exista ningún registro en la tabla Reservas asociado a cada habitación. Dentro de la subconsulta, SELECT 1 comprueba la existencia de registros que coincidan con el identificador de habitación. Si no encuentra ninguno, la habitación aparece en el resultado.

16. **Buscar habitaciones disponibles en un periodo específico**
    Objetivo: buscar habitaciones que no tengan reservas que se superpongan con el período comprendido entre dos fechas.

Utiliza DECLARE para crear dos variables de tipo DATE, donde se guardan las fechas de entrada y salida solicitadas. Luego, NOT EXISTS comprueba que no haya reservas que coincidan con la habitación y se superpongan con ese período. Las condiciones R.FechaEntrada < @FechaSalida y R.FechaSalida > @FechaEntrada permiten detectar los cruces de fechas. De esta manera, solamente se muestran las habitaciones sin reservas superpuestas.

17. **Reservas sin pagos registrados**

    Objetivo: ver que reservas no tienen ningún pago registrado y calcula su coste según su duración y precio por noche

    Utiliza un LEFT JOIN para relacionar las reservas con los pagos, conservando todas las reservas aunque no tengan un pago asociado. DATEDIFF() calcula la cantidad de noches y el resultado se multiplica por PrecioPorNoche. Después, IS NULL filtra los registros donde IdPago no tiene ningún valor, identificando las reservas sin pagos asociados.

18. **Cuanto cobró el hotel por metodo de pago**

    Objetivo: ver cuanto dinero se ingresó según cada método de pago y cuantos pagos se realizaron con cada uno

    Utiliza SELECT para mostrar el método de pago y aplica COUNT() para contar los pagos y SUM() para sumar sus importes. GROUP BY agrupa los registros según el método utilizado. ORDER BY DESC ordena los métodos según el dinero recaudado.

19. **Gastos por categoria**

    Objetivo: calcular cuanto dinero gastó el hotel en cada categoría de gastos y cuántos registros tiene cada una

    Consulta la tabla Gastos y utiliza COUNT() para contar los gastos y SUM() para calcular el monto total de cada categoría. GROUP BY agrupa los gastos según CategoriaGasto y ORDER BY DESC ordena las categorías desde la que representa mayor gasto hasta la menor.

20. **Calcular las horas trabajadas de cada empleado**

    Objetivo: calcular las horas trabajadas por cada empleado según sus registros de asistencia.

    Utiliza un INNER JOIN para relacionar los empleados con sus registros de asistencia mediante IdEmpleado. La función DATEDIFF(MINUTE) calcula los minutos transcurridos entre la hora de entrada y salida. Después, divide el total por 60.0 para convertirlo en horas, utilizando decimales. SUM() acumula las horas y GROUP BY agrupa los resultados por empleado

21. **Consultar estados de problemas del hotel**

    Objetivo: mostrar las solicitudes de mantenimiento del hotel, indicando el empleado responsable, el área afectada, el equipo y el estado de cada solicitud

    Utiliza un INNER JOIN para relacionar las solicitudes de mantenimiento con los empleados que las registraron. El comando WHERE utiliza OR para incluir las solicitudes que tengan alguno de los tres estados indicados: "En Proceso", "En Espera" o "Finalizada".

22. **Ver que servicios adicionales son más usados**
    Objetivo: identificar los servicios adicionales más utilizados por los huéspedes
    Utiliza un INNER JOIN para relacionar los servicios adicionales con sus registros de consumo mediante IdServicio. La función COUNT(*) cuenta cuántas veces aparece cada servicio en los registros

12. Vistas

1. **vw_ReservasDetalle**
   Objetivo: mostrar información detallada de las reservas, incluyendo fechas, estado, cantidad de huéspedes, precio por noche, cantidad de noches, importa total calculado y datos de la habitación asignada. También permite ver reservas correspondientes a un tipo de habitación
   Mediante SELECT selecciona los datos que se quieren mostrar e INNER JOIN relaciona las tablas Reservas, Habitaciones y TiposDeHabitacion utilizando sus identificadores correspondientes. La función DATEDIFF() calcula la cantidad de noches entre la entrada y la salida, mientras que una multiplicación permite calcular el importe total de la reserva.

2. **vw_EstadoCuentaReservas**
   Objetivo: mostrar el estado económico de cada reserva, incluyendo el coste del alojamiento, cuantos pagos se realizaron, cuánto dinero se abonó y el saldo pendiente
   Mediante un LEFT JOIN relaciona las reservas con los pagos, permitiendo incluir también aquellas reservas que todavía no tienen pagos registrados. La función COUNT() cuenta los pagos realizados, mientras que SUM() suma sus importes. Se utiliza COALESCE() para reemplazar los valores nulos por cero cuando no existen pagos

3. **vw_GastosMensuales**
   Objetivo: consultar los gastos del hotel agrupados por año, mes y categoría. Muestra la cantidad de gastos registrados, el dinero total gastado y el promedio de cada gasto
   Mediante YEAR() y MONTH() extrae el año y el mes de la fecha de cada gasto. Después, COUNT() cuenta los gastos registrados, SUM() calcula el total gastado y AVG() obtiene el promedio de los importes.
   Esta vista permite realizar consultas adicionales como mostrar los gastos de un mes o calcular los gastos de cada categoría utilizando un SUM().

4. **vw_ConsumosHuespedes**
   Objetivo: mostrar cuanto dinero gastó cada huésped en productos del hotel y cuantos consumos realizó. Permite identificar que huéspedes tuvieron mayor consumo
   Mediante varios INNER JOIN relaciona las tablas Huespedes, ConsumosDeHuesped, ProductosPorConsumo y Productos. La función COUNT(DISTINCT) cuenta la cantidad de consumos diferentes realizados por cada huésped, evitando contar dos veces un mismo consumo. Por otro lado, SUM() calcula el dinero gastado multiplicando la cantidad de productos consumidos por sus respectivos precios

5. **vw_HistorialEstadias**
   Objetivo: mostrar un historial de las estadías del hotel, incluyendo horas de Check-in y Check-out, estado de cada estadía, su habitación asignada y la cantidad de horas que duró cada una.
   Utiliza un INNER JOIN para relacionar las tablas Estadias y Reservas mediante el identificador de reserva. Para calcular la duración utiliza CASE, que permite establecer diferentes condiciones. Si FechaHoraCheckOut es NULL, significa que todavía no se registró la salida, por lo que calcula las horas transcurridas desde el Check-In hasta el momento actual utilizando GETDATE(). En caso contrario, calcula la diferencia entre el Check-In y el Check-Out mediante DATEDIFF(HOUR).

14.Triggers

1. **TRG_VALIDARDISPONIBILIDADHABITACION**
   Objetivo: Evita que una misma habitación tenga reservas confirmadas cuyas fechas de estadía se superpongan.
   El trigger se ejecuta automáticamente después de realizar una operación de INSERT o UPDATE sobre la tabla Reservas.
   Mediante la tabla virtual inserted, obtiene los datos de las reservas insertadas o modificadas y verifica si existe otra reserva confirmada para la misma habitación con fechas que se superpongan.
   Si detecta un conflicto, utiliza THROW para generar un error e impedir que se confirme la operación, evitando que una habitación sea reservada por distintos huéspedes durante un mismo período.
2. **TRG_VALIDARCANCELACIONRESERVA**
   Objetivo: Impedir que se cancelen reservas fuera del plazo permitido, respetando las reglas de negocio.
   El trigger se va ejecutar después de realizar una operación UPDATE sobre las Reservas.
   Mediante la tablas virtuales inserted y deleted, verifica si una reserva cambió su estado a cancelada.
   Si alguna está cancelada, comprueba si la fecha actual se encuentra dentro del período en el que ya no está permitido cancelar la reserva.
   Si se intenta cancelar una reserva en la fecha de entrada o posteriormente, el trigger utilizar Throw para impedirlo.
3. **TRG_VALIDARCAPACIDADHABITACION**
   Objetivo: evitar que la cantidad de huéspedes asociados a una reserva supere la capacidad máxima permitida para la habitación asignada.
   El trigger se ejecuta después de realizar una operación de insert o update sobre la tabla ReservasHuespedes.
   Mediante la tabla virtual inserted, se identifican las reservas afectadas y se consulta la capacidad máxima.
   Se cuenta la cantidad de huéspedes asociadas a cada reserva y compara ese valor con la capacidad máxima establecida para el tipo de habitación.
   Si se supera, se utiliza el throw.
4. **TRG_VALIDARTITULARRESERVA**

Objetivo: evitar que una misma reserva tenga más de un huésped registrado como titular, garantizando que exista un único responsable principal de la reserva.
El trigger se ejecuta después de realizar una operación INSERT o UPDATE sobre la tabla ReservasHuespedes.
Mediante la tabla virtual inserted, se identifican las reservas afectadas.
Se consulta los huéspedes asociados a cada reserva y se utiliza un count para contar aquellos que tiene el tributo EsTitular en 1
Si se detecta que una reserva tiene más de un huésped como titular, se utiliza un throw.

5. **TRG_VALIDARHORARIOCHECKIN**
   Objetivo: Impedir que los huéspedes realicen el check-in-antes de las 14:00 horas, respetando el horario de ingreso establecido por el hotel.
   El trigger se ejecuta después de una operación de insert o update sobre la tabla Estadías
   Mediante la tabla virtual inserted, se obtiene la fecha y hora del check-in de los registros.
   Después se utiliza un cast para obtener únicamente la hora del atributo FechaHoraCheckIn y comprobar si es anterior de las 14:00 horas.
   Si se detecta que el horario de ingreso es anterior al permitido se utiliza el throw.
6. **TRG_VALIDAREDADTITULAR**
   Objetivo: impedir que un huésped menor de 18 años sea registrado como titular de una reserva, garantizando el cumplimiento de la edad mínima.
   El trigger se ejecuta después de realizar una operación insert o update sobre ReservasHuespedes
   Mediante una tabla virtual inserted, obtiene las fechas de nacimiento de cada huésped.
   Después verifica cuales huéspedes tiene el atributo EsTitular en 1 y utiliza la función DATEADD para calcular la fecha en la que cumplen 18 años.
   Esa fecha se compara con la fecha actual para determinar si el huésped alcanzó la mayoría de edad.
   Si detecta que un huésped registrado como titular no cumplió los 18 años, se utiliza el throw.
7. **TRG_ACTUALIZARSTOCK**
   Objetivo: Actualizar automáticamente el stock de los productos consumidos por los huéspedes, evitando que se registren consumos superiores a las existencias disponibles.
   El trigger se ejecuta después de realizar una operación insert sobre la tabla ProductosPorConsumo.
   Mediante una tabla virtual inserted, obtiene los productos y las cantidades registradas de los nuevos consumos.
   Después se utiliza una función sum junto con group by para calcular la cantidad total consumida de cada producto.
   También se realiza un inner join con la tabla Productos para comparar las cantidades consumidas con el stock disponible.
   si se detecta que se supera la existencia, se utiliza throw.
8. **TRG_ALERTASTOCKBAJO**
   Objetivo: detectar cuanto el stock de un producto se encuentra por debajo del nivel mínimo establecido.

El trigger se ejecutar despues de realizar una operación update sobre la tabla Productos
Mediante la tabla virtual inserted, obtiene los valores actualizados del stock de los productos.
Después se verifica si alguno de los productos tiene una cantidad disponible inferior a 10 unidades, tomando como referencia un stock inicial de 100 unidades.
Si detecta que un producto se encuentra por debajo de ese límite, se utiliza el print para mostrar un mensaje de advertencia.

9. **TRG_ActualizarEstadoEstacionamiento**
   Objetivo: Actualizar el estado de una plaza de estacionamiento cuando se le asigna un vehículo, evitando que una misma plaza sea ocupada por más de un vehículo.
   El trigger se ejecuta después de realizar una operación insert sobre la tabla VehículosPlazas.
   Mediante la tabla virtual inserted, identifica las plazas de estacionamiento a las que se les asignaron vehículos.
   Después se consulta la tabla VehiculosPlaza y se utiliza group by junto having count(*) > para comprobar si alguna de las plazas tiene más de un vehículo.
   Se detecta que una plaza tiene más de un vehículo, se utiliza throw.
   En caso de que la asignación sea válida, se ejecuta un update sobre la tabla PlazaDeEstacionamiento, modificando el atributo Estado a “ocupada”

10. **TRG_LIBERARPLAZAESTACIONAMIENTO**
    Objetivo: actualizar el estado de un plaza de estacionamiento cuando se elimina la asignación de un vehículo, permitiendo que vuelva a estar disponible para otros huéspedes.
    El trigger se ejecuta después de una operación delete sobre la tabla VehiculosPlaza mediante la tabla virtual deleted, identifica las plazas de estacionamiento que estaban asociadas a los vehículos cuyas asignaciones fueron eliminadas.
    Después se consulta la tabla VehiculasPlaza para verificar si las plazas afectas todavía tiene algún vehículo asignado.
    Para realizar esta comprobación se utiliza un NOT EXISTS que permite identificar aquellas plazas que ya no tiene ninguna asignación registrada.
    Si una plaza no tiene vehículos asociados, se ejecuta un update sobre la tabla PlazasDeEstacionamiento, modificando el atributo Estado a “Disponible”.

15.Cursores

1.**1. calcGastoProm**

Objetivo: calcular cuántos días estuvo alojado cada huésped, cuánto gastó en productos del hotel y cuánto gastó por día
El cursor se declara con DECLARE CURSOR FAST_FORWARD y recorre la tabla Huespedes de a un huésped por vez. Con OPEN lo abre y con FETCH NEXT guarda el IdHuesped en la variable @IdHuesped. Dentro de un WHILE que se repite mientras @@FETCH_STATUS sea 0, un SELECT suma los días de las estadías del huésped, se relaciona ReservasHuespedes con Estadias con INNER JOIN. Si la estadía no tiene check-out, usamos un ISNULL() que deja la fecha actual. Luego otro SELECT calcula lo

gastado con SUM(), multiplicando la cantidad consumida por el precio de cada producto, mediante INNER JOIN entre ConsumosDeHuesped, ProductosPorConsumo y Productos. Después utiliza IF para comprobar si los días son cero. Si lo son, pone el gasto por día en cero para evitar una división por cero. En caso contrario, divide el gasto total por la cantidad de días

2. masPlataxFactEmpleado

Objetivo: encontrar el empleado que emitió facturas por el mayor importe total
El cursor se declara con DECLARE CURSOR FAST_FORWARD y recorre la tabla Empleados de a un empleado por vez. Antes de abrirlo se crean las variables @maxTotal con valor -1 para guardar el mayor total encontrado y @IdEmpleadomax para guardar el empleado que lo logró. Con OPEN abre el cursor y FETCH NEXT guarda el IdEmpleado en una variable. Dentro de un WHILE que se repite mientras @@FETCH_STATUS sea 0 un SELECT calcula el total facturado por el empleado con SUM() multiplicando PrecioUnitario por Cantidad y sumándole el Impuesto. Usa INNER JOIN para relacionar Facturas con DetallesFactura y usa un where para sacar los que no sirven. Después con un IF compara el total del empleado con @maxTotal, si es mayor o igual actualiza @maxTotal y @IdEmpleadomax con los datos de ese empleado. Al finalFETCH NEXT pasa al siguiente

3. agregarCostomulta

Objetivo: aumentar un 10% el costo de las reservas cuya estadía terminó después de la fecha de salida que tenían prevista
El cursor se declara con DECLARE CURSOR FAST_FORWARD y recorre la tabla Reservas de a una reserva por vez. Con OPEN abre el cursor y FETCH NEXT guarda IdReserva. Dentro de un WHILE que se repite mientras @@FETCH_STATUS sea 0 un SELECT obtiene la fecha real de salida (FechaHoraCheckOut) de la tabla Estadias, que se relaciona con Reservas con INNER JOIN, y otro SELECT obtiene la fecha de salida prevista (FechaSalida) de la reserva. Antes de cada consulta las dos variables se ponen en NULL porque sino quedan con datos de la reserva anterior. Después con un IF compara las dos fechas, si la real es mayor que la prevista hace un UPDATE sobre Reservas que multiplica PrecioPorNoche por 1.10. Al final FETCH NEXT pasa a la siguiente reserva

4. actualizarHabitaciones

Objetivo: actualizar el estado de cada habitación según las estadías y las reservas que tiene el cursor se declara con DECLARE CURSOR FAST_FORWARD y recorre la tabla Habitaciones de a una habitación por vez. Con OPEN abre el cursor y FETCH NEXT guarda el IdHabitacion en una variable. Dentro de un WHILE que se repite mientras @@FETCH_STATUS sea 0 un SELECT con COUNT() cuenta las estadías con estado En curso de la habitación, relacionando Estadias con Reservas con INNER JOIN, y se guarda en la variable @ahora. Otro SELECT con COUNT() cuenta las reservas de esa habitación con fecha de entrada posterior a la actual y con estado Confirmada o Pendiente, y se guarda en @futuras. Después utiliza IF para decidir el estado, si @ahora es mayor a cero hace un UPDATE y pone la habitación como Ocupada, si no y @futuras es mayor a cero la

pone como Reservada y si no se cumple ninguna la deja como Disponible. Al final FETCH NEXT pasa a la siguiente habitación

5. limpiarHabitaciones

Objetivo: crear una tarea de limpieza para cada habitación que tiene la limpieza pendiente y asignarla a un empleado que esté libre
El cursor se declara con DECLARE CURSOR FAST_FORWARD y recorre la tabla Habitaciones de a una habitación por vez. Antes de abrirlo se crea una tabla variable @empleadoLibre como tabla para guardar lo que devuelve el procedimiento almacenado sp_EmpleadoLimpiezaLibre porque este devuelve una tabla. Con OPEN abre el cursor y FETCH NEXT guarda el IdHabitacion en una variable. Dentro de un WHILE que se repite mientras @@FETCH_STATUS sea 0 un SELECT obtiene el estado de limpieza de la habitación y con un IF comprueba si es Pendiente. Si lo es, con DELETE vacía la tabla variable para que no queden datos de la vuelta anterior, ejecuta el procedimiento con INSERT ... EXEC y con un SELECT guarda el IdEmpleado en la variable @empleado. Luego hace un INSERT en LimpiezasdeHabitaciones con la hora de inicio GETDATE(), la hora de fin una hora después con DATEADD(HOUR), el empleado y la habitación. Después un UPDATE cambia la limpieza de la habitación a Limpia. Al final FETCH NEXT pasa a la siguiente habitación

16.Funciones

1. **fn_ImporteReserva**
   Objetivo: calcular el importe de una reserva específica, teniendo en cuenta el precio por noche y la cantidad de noches que se aloja
   La función recibe como parámetro @IdReserva, un identificador de reserva de tipo entero (INT). Mediante RETURNS DECIMAL(12,2) establece que devolverá un valor numérico con dos decimales. Dentro de la función, DECLARE crea una variable llamada @Importe para almacenar el resultado. Luego, SELECT obtiene el precio por noche y lo multiplica por la cantidad de noches calculada con DATEDIFF(DAY). El resultado se guarda en la variable mediante un = y finalmente RETURN devuelve el importe calculado.

2. **fn_SaldoPendienteReserva**
   Objetivo: calcular cuanto dinero le queda por pagar al huésped de una reserva
   La función recibe como parámetro el identificador de una reserva. Mediante DECLARE crea tres variables para almacenar el importe total de la reserva, el total pagado y el saldo pendiente. Primero llama a fn_ImporteReserva() para obtener el costo total del alojamiento. Después, utiliza SELECT, SUM() y COALESCE() para calcular cuánto dinero se pagó, reemplazando por cero el resultado cuando no existen pagos registrados. Luego utiliza IF para comprobar si la reserva existe y tiene un importe calculado. Si el importe es NULL, devuelve NULL. Después resta el total pagado al importe de la reserva. Otra condición IF comprueba si el resultado es negativo y, de ser así, establece el saldo en cero. Finalmente, RETURN devuelve el saldo pendiente.

3. **fn_TotalGastadoHuesped**
   Objetivo: calcular cuánto dinero gastó un huésped específico en los productos consumidos dentro del hotel
   La función recibe como parámetro @IdHuesped. Mediante RETURNS DECIMAL(12,2) establece que devolverá un importe con dos decimales. Dentro de la función, DECLARE crea una variable para almacenar el total gastado. Luego, utiliza un INNER JOIN para relacionar las tablas de consumos, productos por consumo y productos. La función SUM() suma el resultado de multiplicar la cantidad consumida por el precio de cada producto. Mediante WHERE filtra únicamente los consumos correspondientes al huésped indicado. COALESCE() devuelve cero si no existen consumos registrados. Finalmente, RETURN devuelve el total calculado.
4. **fn_HabitacionesDisponibles**
   Objetivo: consultar qué habitaciones están disponibles en un período de tiempo determinado.
   La función recibe dos parámetros de tipo DATE: @FechaEntrada y @FechaSalida y utiliza RETURNS TABLE, lo que significa que devuelve una tabla con varios registros y columnas. Mediante RETURN ejecuta una consulta que utiliza un INNER JOIN para relacionar las habitaciones con sus tipos correspondientes. En el WHERE se establece que la fecha de entrada debe ser anterior a la fecha de salida y utiliza NOT EXISTS para descartar las habitaciones que tengan reservas superpuestas con el período solicitado. La subconsulta compara las fechas de las reservas con los parámetros recibidos para determinar si una habitación ya está ocupada durante ese intervalo.

17.Manejo De Errores

Se incorporan mecanismos de manejo de errores con el objetivo de detectar operaciones inválidas, controlar situaciones y mantener la consistencia de los datos. Para ello, se utilizan estructuras de control, validaciones de parámetros y transacciones, permitiendo identificar problemas antes de realizar operaciones.
Mecanismo utilizados:
TRY..CATCH: permite controlar errores que ocurren durante la ejecución de procedimientos almacenados. El bloque try contiene las operaciones que se desean realizar, mientras que el bloque catch permite tratar los errores que se producen en el try.
THROW: se utiliza para generar errores personalizados cuando se incumplen determinadas condiciones o reglas de negocio. También permite volver a lanzar un error capturado.
Validación de parámetros: permite comprobar que los datos recibidos sean correctos antes de realizar operaciones sobre las tablas.
Transacciones: permiten agrupar varias operaciones que deben ejecutarse como una unidad. Mediante BEGIN TRANSACTION se inicia la transacción, COMMIT confirma los cambios cuando la ejecución es correcta y ROLLBACK deshace los cambios no confirmados cuando ocurre un error.
Estos mecanismos se aplican tanto en trigger como en procedimientos almacenados.

Manejo de errores en el registro de pagos

Procedimiento almacenado: sp_RegistrarPago
Objetivo: Evitar el registro de pagos con datos inválidos o asociados a reservas inexistentes.

El procedimiento utiliza TRY..CATCH para controlar los errores que se puedan producir. También se realiza una validación antes de la inserción, se comprueba que la reserva exista, que el importe no sea nulo ni menor a cero y que cantidad de cuotas sea mayor a cero y no nula.
Si no se cumple la condición, se ejecuta el throw.
En caso de que se produzca un error, el bloque catch lo vuelve a lanzar mediante throw.

Manejo de errores en la asignación de estacionamiento

Se implementó bloques TRY..CATCH en el procedimiento almacenado sp_AsignarEstacionamiento y también transacciones para controlar los errores durante la asignación de una plaza a un vehículo.
Primero, se verifica que existan las condiciones necesarias para la asignación como la disponibilidad de una plaza.
Si se cumple las validaciones, se registra la asignación y se actualiza el estado de la plaza
En caso de que se produzca un error, se utiliza rollback para deshacer los cambios de la transacción y throw para informar el problema. Si la operación finaliza correctamente, se confirma con una commit.

Manejo de errores en el check-in de huéspedes.

Se implementó bloques TRY…CATCH y transacciones en el procedimiento almacenado sp_CheckInEstadia para controlar los errores durante el registro de una estadía.
Primero, se verifican las condiciones necesarias para realizar el check in, como la disponibilidad de una habitación.
Si se cumplen las validaciones, se registra la estadía y actualiza la información correspondiente a la habitación y la reserva.
En caso de que se produzca un error, se utiliza un ROLLBACK para deshacer los cambios y throw para informar el problema.
Si la operaciones finalizan correctamente, se utiliza commit para confirmar los cambios.

18.Seguridad de la Base de Datos

Objetivo: controlar quién entra a la base de datos del hotel y qué puede hacer cada uno, dándole a cada actor solo lo que necesita para su trabajo

Primero se crean los esquemas con CREATE SCHEMA, que agrupan a los usuarios según su función. Después con CREATE LOGIN se crean las cuentas para entrar al servidor, cada una con su contraseña, y con CREATE USER el usuario dentro de la base GestionDeHotel, unido a su login. Con ALTER USER WITH DEFAULT_SCHEMA le asignamos a cada usuario su esquema. Los permisos se manejan con GRANT para dar un permiso y DENY para prohibirlo de forma explícita.

Esquemas

Creamos cinco: admin, gerente, recepcionista, personal y huesped. El esquema personal lo comparten limpieza, mantenimiento, estacionamiento, contable, inventarios, seguridad y servicios extra, porque no hace falta uno para cada uno

Logins y usuarios

Hay un login y un usuario por cada actor, por ejemplo login_admin y usuario_admin, login_gerente y usuario_gerente, y así con el resto. Admin, gerente recepcionista y huésped usan su propio esquema por defecto y los demás usan personal

Permisos por actor

1. Administrador
   Objetivo: manejar toda la base de datos
   Tiene GRANT de SELECT, INSERT, UPDATE, DELETE y EXECUTE sobre todo el esquema
2. Gerente
   Objetivo: ver toda la información del hotel sin modificarla
   Con GRANT puede hacer SELECT sobre todo y ejecutar las funciones con EXECUTE. Con DENY no puede hacer INSERT, UPDATE ni DELETE, así no cambia los registros del día a día
3. Recepcionista
   Objetivo: atender reservas, check-in y check-out
   Puede hacer SELECT, INSERT y UPDATE sobre Reservas y Estadias, y select e INSERT sobre Huespedes y ReservasHuespedes. También puede ver Habitaciones y TiposDeHabitacion, consultar fn_HabitacionesDisponibles y ejecutar fn_ImporteReserva. Con DENY no puede tocar Pagos, Facturas ni Gastos porque son del área contable, y tampoco borrar reservas ni estadías.
4. Personal de limpieza
   Objetivo: registrar las limpiezas y actualizar el estado de las habitaciones
   Puede hacer SELECT, INSERT y UPDATE sobre LimpiezasdeHabitaciones, y SELECT y UPDATE sobre Habitaciones para cambiar el estado de limpieza.
5. Personal de mantenimiento
   Objetivo: manejar las solicitudes de mantenimiento
   Puede hacer SELECT, INSERT y UPDATE sobre SolicitudesDeMantenimiento.
6. Personal de estacionamiento
   Objetivo: controlar qué plazas están ocupadas
   Puede hacer SELECT sobre Vehiculos, y SELECT, INSERT y UPDATE sobre PlazasDeEstacionamiento y VehiculosPlazas. No puede modificar los datos de los vehículos
7. Personal contable
   Objetivo: manejar facturas, pagos y gastos
   Puede hacer SELECT, INSERT y UPDATE sobre Facturas, DetallesFactura, Pagos y Gastos. Puede ver Reservas y la vista vw_EstadoCuentaReservas, y ejecutar fn_SaldoPendienteReserva. No puede modificar reservas
8. Jefe de inventarios
   Objetivo: controlar el stock y los precios de los productos
   Puede hacer SELECT, INSERT y UPDATE sobre Productos y SELECT sobre Categorias
9. Personal de seguridad
   Objetivo: saber quiénes están alojados en el hotel
   Puede hacer SELECT sobre Huespedes, Reservas y ReservasHuespedes. Con DENY no puede hacer INSERT UPDATE ni DELETE sobre Huespedes ni Reservas.
10. Personal de servicios extra
    Objetivo: registrar los servicios extra que usan los huéspedes
    Puede hacer SELECT, INSERT y UPDATE sobre ServiciosExtra y ConsumosServiciosExtra, y SELECT sobre Estadias para saber a qué estadía va cada consumo

11. Huésped
    Objetivo: que alguien de afuera no vea datos internos del hotel
    No tiene permisos sobre las tablas. Con DENY no puede ver Empleados, Pagos ni Gastos, y tampoco borrar reservas